Iteration 5 — Change Report

The behaviour tree fix holds in 14 of 14 episodes, the 100×100 m cost map keeps its rate, and run 8's collision has a complete and alarming explanation

Summary

The two changes you most wanted tested both passed

The behaviour-tree restructure works. Fourteen recovery episodes across three bags, and every single one opened with the unconditional `Wait` for 2.02 s. Clearing was reached three times and on all three occasions only after *both* `BackUp` rungs had returned FAILURE in that same episode. The leaked index is still there and is now harmless — it only chooses whether 0.30 m or 0.50 m is reversed first. Section 1.

The 100×100 m global cost map holds its configured rate. 6.25 million cells, and the update loop ran at a median of 500.0 ms against a configured 2 Hz, with no missed cycles. The thing that got worse is not the loop, it is the planner's tail. Section 2.

Run 8 is the most important thing in this iteration, and it was not a nav2 bug

The robot stalled 1.02–1.15 m short of a goal whose tolerance is 1.00 m, with the last bin between it and the goal. At that range the bin is **below every LiDAR ring**, so it was never re-observed. Recovery cycled four times over 27 s. Then STVL's 30 s decay erased the bin, the planner immediately returned a path straight through where it had been, and the robot drove into it.

Nothing cleared the cost map. No `ClearEntireCostmap` ran in that bag at all. This was decay alone, and it is the scenario from your whiteboard drawing happening at 1 m instead of across the course. Section 3.

Two defects nobody reported, both found in the logs

The base controller was throwing away commands. Every gap in the nav2 command stream produced exactly 10 rejected velocity commands. 100 of them in run 10. Section 7.

A stuck robot with a valid plan waits 20 s before anything happens. That is the "sitting there without recovery" you asked about, and it has a one-line cause. Section 6.

Ten bags, 762 s of running, 21 GB. Runs 1, 5 and 9 produced **zero** warnings of any kind, which has not happened in any previous iteration.

1. How well did the behaviour tree changes work?

Completely. This is the cleanest result in the set, so it is worth showing the evidence rather than asserting it.

Every episode opened with the wait

Fourteen episodes, defined as a `Wait` going IDLE → RUNNING at the top of the recovery branch:

bag	episodes	opened with Wait	wait duration	clears
real_bag2	2	2 of 2	2.02 s both	2
real_bag8	4	4 of 4	2.02 s all	0
real_bag10	8	8 of 8	2.02 s all	1

bag	episodes	opened with Wait	wait duration	clears
total	**14**	**14 of 14**	**2.02 s**	**3**

In iteration 4 the wait happened on whichever episodes the rotating index happened to land on it. It now happens on all of them, because it is a `Sequence` child rather than a round-robin rung, and `SequenceNode` resets its own child index when it returns.

Clearing was reached only on exhaustion, three times

All three clears look like this — two `BackUp` FAILUREs, then the round robin itself returning FAILURE, then the clear, in the same tick (from run 10, episode 4):

```
58.89  BackUp                          IDLE -> RUNNING
62.31  BackUp                          RUNNING -> FAILURE
62.31  BackUp                          IDLE -> RUNNING
62.33  BackUp                          RUNNING -> FAILURE
62.33  RecoveryActions                 IDLE -> FAILURE
62.33  ClearLocalCostmap-LastResort    IDLE -> SUCCESS
```

That is `RoundRobinNode` requiring `num_failed_children_` to reach the child count. It is a semantic last resort now, not a positional one. The failing reverses were the familiar error 714, `Collision Ahead - Exiting DriveOnHeading` — 4 in run 2, 2 in run 10.

The index leak is visible, and so is halt() resetting it

The two reverses take measurably different times, so the log says which rung ran: 0.30 m at 0.10 m/s is about 3.4 s and 0.50 m is about 5.4 s. Run 10's eight episodes:

```
ep1 3.42 s   ep2 5.41 s   ep3 3.32 s   ep4 FAILED both -> CLEAR
ep5 3.42 s   ep6 5.42 s   ep7 3.42 s   ep8 5.22 s
```

Episodes 1–3 alternate 0.30 / 0.50 / 0.30, exactly as a leaking index predicts. Episode 4 exhausts the round robin, which calls `halt()`, which sets `current_child_idx_ = 0` — and episode 5 duly starts at 0.30 again and the alternation restarts cleanly. Run 8's four episodes alternate 3.42 / 5.52 / 3.42 / 5.42 with no clear and therefore no reset. The mechanism is confirmed in both directions.

> **What is still not verified**
>
> Nothing in these bags exercises the case where a reverse is vetoed in an episode that *began* at the 0.50 m rung and then succeeded at 0.30 m. It should work by the same logic, but it has not been observed.

2. What did the loop rates look like?

Measured off `published_footprint`, which `Costmap2DROS::updateMap()` emits unconditionally once per update-loop iteration. It is the loop's heartbeat and it is not gated by `publish_frequency`, so it separates "did the map update" from "did the map get published" — a distinction that matters here.

what	cells	configured	measured median	p90	max
local update loop	160 k	10 Hz	**100.0 ms**	100.2–106.1 ms	109.8 ms
global update loop	**6.25 M**	2 Hz	**500.0 ms**	500.5–506.4 ms	530–600 ms
local /costmap publish	—	10 Hz	197 ms (6.4 Hz)	200.2 ms	235 ms

what	cells	configured	measured median	p90	max
global /costmap publish	—	5 Hz → 2 Hz	500 ms	501.8 ms	1024–1078 ms

The headline is that the big map is free. A 2500×2500 cell global cost map at 4 cm held 2 Hz with no missed cycles in any of the ten runs. The worst single global cycle in the whole set was 600 ms against a 500 ms budget. Going to 100×100 m did not cost you the update loop.

Two smaller things fell out of this. The local `/costmap` topic was publishing at 6.4 Hz from a 10 Hz loop, because `publish_frequency` equalled `update_frequency` and the gate in `Costmap2DROS` is a strict less-than, so it fails on every other cycle — a 45/55 split between 100 ms and 200 ms gaps. I have set it to 20.0, which makes the publish period strictly shorter than the loop period so the gate passes every cycle. It cannot publish faster than the loop runs, so this buys an honest, even 10 Hz for nothing. And the global `publish_frequency: 5.0` is simply unreachable: publishing is tested once per update loop, so 2 Hz is the ceiling. It is harmless but it reads as though something is happening at 5 Hz. The occasional 1.0 s gap on the global topic is a skipped publish of a 6.25 MB message, not a skipped update.

Planning is where the big map actually shows up

bag	plans	median	p90	max	failures	at/over the 450 ms budget
real_bag1	58	69.5 ms	90 ms	100 ms	0	0
real_bag2	93	70.3 ms	100 ms	**869 ms**	2	**2 of 2**
real_bag8	75	20.1 ms	41 ms	610 ms	4	0 (all 20 ms)
real_bag10	39	79.3 ms	**482 ms**	**689 ms**	9	**6 of 9**

The median is fine everywhere. The tail is not, and it splits the two failed runs apart cleanly:

• **Run 10's failures are timeouts.** Median 480 ms against a 450 ms budget, six of nine at or over it. The planner is not finding the route blocked, it is running out of time looking. A blocked query on a 100×100 m map fans out across far more free space before giving up than the same query on 30×30 m did.

• **Run 8's failures are genuine.** All four returned in 20.1 ms. That is the planner saying "there is no route" immediately and being right.

> `max_planning_time` **was not being enforced**
>
> `terminal_checking_interval` was 5000, meaning the clock is only read once every 5000 node expansions. So the planner overshoots its budget by however long the next 5000 expansions take — measured at **up to 1.9×**: 689 ms and 869 ms against a 450 ms budget.
>
> I have set it to 1000. This **lowers worst-case planning latency**, which also means the tree learns about a planning failure sooner. It will not make any query succeed that failed before. I deliberately did *not* raise the budget itself — see section 10.

Replanning delivered 1.03–1.08 s intervals against the new `RateController hz=1.0`, so that change took effect as intended. At a 70 ms median that is a 7% duty cycle; the replan rate is still not the constraint.

3. What happened in run 8?

This is worth reading in full, because every step is measured and the conclusion is not comfortable.

The sequence

t	what happened	evidence
27.7 → 47.7 s	Robot stuck but with a valid plan. Nothing tries to recover.	`Failed to make progress` (error 105) fires at 47.7 s, so the progress checker's last reset was 20.0 s earlier. See section 6.
67.7 s	Robot reaches 1.15 m from the goal and the planner starts failing.	`GridBased plugin failed to plan from (6.49, -1.54) to (7.48, -2.13)` — a separation of 1.152 m, against `xy_goal_tolerance: 1.0`.
67.7 → 100.8 s	Four recovery episodes. Each one waits 2 s, reverses successfully, drives forward again, and ends up in the same place.	Distance to goal cycles 1.02–1.06 m → 1.40 m → 1.02 m, four times. Reverses took 3.42 / 5.52 / 3.42 / 5.42 s and all four SUCCEEDED.
93.8 → 95.4 s	**The bin is erased from the cost map while the robot stares at it.**	Lethal cells on the line to the goal go **94** → **72** → **39** → **0** and the peak cost falls **254** → **223** → **190**. Robot stationary at `vx = 0.00` throughout.
95.4 s onward	Planner finds a path straight through where the bin was. Robot drives forward and hits it.	Lethal count stays at 0 for the rest of the run.

Why the bin was never re-observed

From `/tf_static` in this bag, the Velodyne sits 1.225 m above the ground. With a VLP-16's ±15° spread, the lowest ring's height against range is:

```
range   lowest ring   anything shorter is below EVERY ring
 0.5 m       1.09 m    < 1.09 m  invisible
 1.0 m       0.96 m    < 0.96 m  invisible
 1.5 m       0.82 m    < 0.82 m  invisible
 3.0 m       0.42 m    < 0.42 m  invisible
```

The robot was stalled with the bin between 0.5 m and 1.0 m away. A bin is nowhere near 0.96 m tall. It was **completely invisible for the entire 27 s**, while sitting squarely inside the sensor frustum — which is the worst combination, because STVL's `decay_acceleration` only applies in-frustum. The mark aged out at roughly 30 s after its last observation, matching `voxel_decay: 30.0` directly.

The part that is not nav2's fault

The goal was about 1 m behind the final bin. The robot's front face is only 0.230 m ahead of `base_link`, so for the robot's *centre* to get within 1.00 m of that goal, its front has to be roughly 0.77 m from it — which is inside the bin. **The goal was not reachable to within its own tolerance.** The planner returning 208 in 20 ms was correct, and no amount of reversing was ever going to fix it.

It was also very close. The robot got to 1.02 m with a 1.00 m tolerance. It lost the run over 2 cm.

What I changed for this, and what I did not

Changed: `voxel_decay` 30 → 60 s on both STVL layers, and `number_of_retries` on `NavigateRecovery` 8 → 4. These two numbers have to be set against each other, and that is the point: four episodes took 33 s in this bag, so the recovery loop now gives up roughly **25 s before the map forgets**, instead of still running when it does.

Not changed: the real defect is that the robot will drive into space it cannot currently see, and no parameter fixes that. Raising the decay to 60 s moves the cliff from 27 s to 57 s, it does not remove it. Section 10 has the structural options.

Watch for: longer memory is what caused the iteration-1 failure, where marks piled up until no route existed. This is the change in this iteration most likely to misbehave.

4. What happened in run 2 at the corridor entry?

The oscillation is the plan moving, not the controller chattering. Three measurements say so:

measurement	value	reading
Commanded `wz` sign flips	31 in 81.1 s = **0.38/s**	Slow weaving, not chatter. Chatter would be several per second.
Dominant `wz` frequency	**0.16 Hz** (6.09 s period), 8% of in-band power	Far too slow to be a delay-driven limit cycle, which would sit near 1.25 Hz for the measured lag.
Tracking lag, command → achieved	**220 ms** wheels, **200 ms** gyro, gain 0.99, correlation 0.99	The robot tracks what it is told, almost perfectly, 0.2 s later. The loop is not fighting itself.
Plan heading at 1 m, between replans	median **5.1°**, p90 **15.0°**, max **23.7°** over 90 replans	**This is the oscillation.** The robot is faithfully chasing a reference that moves 15° once a second.

So the robot weaves because successive global plans disagree about which side of the corridor to use, and at `hz=1.0` it has a full second to commit to each answer before the next one arrives. Halving the replan rate this iteration made each disagreement bigger and gave the robot longer to act on it, which is consistent with the oscillation you noticed getting worse.

Why the plans disagree is section 5: in a corridor this narrow the cost field is nearly flat, so left-leaning and right-leaning routes score almost the same and the planner flips between near-ties. `change_penalty: 0.00` means nothing resists the flip.

The slow-down before the corridor and speed-up inside it is the same story from the other side. Approaching the mouth, the robot can see both bin edges and the walls, the cost field ahead is at its most ambiguous, and MPPI's surviving samples are the slow ones. Once inside, the geometry is settled, the plan stops flipping, and speed recovers.

The 200–220 ms tracking lag is worth a separate look. It is larger than the 40–55 ms of actuation dead time measured in iteration 3, and it appears on the wheel odometry as well as the gyro, so it is not an artifact of dropping the IMU to 40 Hz. I have not chased it down and it is not obviously hurting anything at these speeds, but it should not be 4× the earlier figure.

5. What specifically slows MPPI down?

Two things, and the first one is geometry rather than a setting.

The cost field has no free space in it

The inflation layer assigns `253 · exp(-cost_scaling_factor · (d - inscribed_radius))` to a cell a distance *d* from the nearest lethal cell. With `cost_scaling_factor: 1.7` and an inscribed radius of 0.230 m:

distance from the nearest obstacle	predicted cost	what sits there
0.80 m (centre of a 1.6 m corridor)	**96**	the best cell available anywhere in the corridor
0.49 m	163	the robot's own side, if perfectly centred
0.315 m	219	the robot's side, slightly off centre

Measured cost under the robot's centre cell across these runs: 64–165, median around 120. Measured cost on the footprint outline: median 249–253. The model and the bags agree to within a few counts.

So there is no zero-cost cell anywhere in the corridor. MPPI's `CostCritic` accumulates `traj_cost += pose_cost` at every sampled point of every rollout using the *centre-cell* cost, so every one of the 6000 trajectories carries a large and very similar baseline. With `temperature: 0.3` the softmax is choosing between near-identical scores, which both flattens the gradient MPPI is steering by and makes the global planner's route choice a coin flip — which is section 4.

Note the asymmetry worth knowing: `CostCritic` *collision-checks* with the footprint but *scores* on the centre cell only. And `near_collision_cost: 253`, which adds `critical_cost: 300`, therefore fires on the centre cell, not the outline — so it almost never fires, even though the outline is at 253 for 49% of run 1, 87% of run 8 and 49% of run 10.

The sample cloud does not fit the corridor

speed	1σ yaw sample	lateral excursion over the 3.0 s horizon	fits a 0.53 m clearance?
1.0 m/s	0.45 rad/s	**1.74 m**	**no**
0.5 m/s	0.45 rad/s	**0.87 m**	**no**
0.2 m/s	0.45 rad/s	**0.35 m**	yes

The horizon is `time_steps: 30` × `model_dt: 0.1` = 3.0 s. With `wz_std: 0.45`, a one-sigma yaw sample swings the rollout well outside the corridor at any speed above a crawl — and in run 8 the measured clearance on the robot's left was **0.04 m** beyond its own footprint (free corridor 0.53 m left + 1.08 m right = 1.61 m, robot 0.98 m wide). Fast samples collide, get `collision_cost: 1000000`, and are annihilated in the softmax. The only survivors are slow and nearly straight.

So MPPI crawling in a tight corridor is largely correct behaviour, not a misconfiguration. What it chose, against a governor limit of 0.90–1.00 m/s:

run	governor limit	MPPI median while moving	MPPI max	average overall
real_bag1	0.90–0.96 m/s	0.351 m/s	0.765 m/s	0.44 m/s
real_bag2	0.90–1.00 m/s	0.487 m/s	0.780 m/s	0.50 m/s
real_bag3	0.65–0.95 m/s	0.257 m/s	0.766 m/s	0.31 m/s
real_bag9	—	0.227 m/s	0.726 m/s	0.27 m/s
real_bag10	0.70–1.00 m/s	0.197 m/s	0.382 m/s	0.15 m/s

Nothing ever reached 0.8 of the cap. The cap is not the constraint and raising it will do nothing. Run 3's "going very slowly" is 0.257 m/s median, and run 9 — the clean sweep — was 0.227 m/s. The runs that worked were the slow ones.

6. How do we get into recovery faster?

The honest answer is that in 13 of the 14 episodes you already do. Measured time from the robot going quiet to the recovery branch opening:

trigger	how it reaches recovery	measured latency
Planner returns 208	The tree sees FAILURE on the next tick.	**1.15–8.8 s**, typically 1.2–1.8 s. This is already fast, and the slow end is the planner's own timeout tail — which section 2's change improves.
Controller cannot make progress	Only when `SimpleProgressChecker` gives up.	**20.0 s, exactly once** — run 8 at t=47.7 s.

So the thing you saw is the second row, and it has a single cause: `movement_time_allowance: 20.0`. The robot must fail to cover `required_movement_radius: 0.35` for a full twenty seconds before `FollowPath` aborts and the tree can reach the recovery branch at all. Until that fires, nav2 believes everything is fine: the plan is valid and the controller is dutifully publishing commands.

I have set it to **8.0 s**. That still asks for only 0.35 m / 8 s = 0.044 m/s, and the slowest sustained speed measured anywhere in these ten runs was 0.197 m/s, so there is a 4× margin before this fires on a robot that is legitimately crawling. I would not take it much below 8 s without re-measuring that margin — and note that lowering `required_movement_radius` would make detection *slower*, not faster, because a creeping robot would keep resetting the clock.

7. The base controller was discarding velocity commands

Nobody reported this. It appears 190 times in run 10's log alone:

```
x100 WARN diffbot_base_controller  Ignoring the received message (timestamp ...)
                                   because it is older than the current time by
                                   0.5412296850 seconds
x55  WARN diffbot_base_controller  Velocity command timed out. Braking.
```

The chain is exact, not approximate. `velocity_smoother` republishes on its own 20 Hz clock but carries its *input's* header stamp. `diff_drive_controller` rejects any command stamped older than `cmd_vel_timeout: 0.5`. The smoother's `velocity_timeout` was 1.0 s. So from 0.5 s to 1.0 s after the last real command, the smoother emits messages the base throws away, and the base then brakes on its own watchdog instead of following the smoother's ramp.

bag	gaps in `/cmd_vel_raw`	smoother outputs per gap	rejected per gap	total rejected
real_bag2	3	24	**10**	30
real_bag8	4	24	**10**	50
real_bag10	9	24	**10**	100

Exactly 10 every time, because 0.5 s at 20 Hz is 10 messages. `/cmd_vel_raw` itself was clean throughout, median stamp age 0.1–0.2 ms — this is created entirely inside the smoother. And every one of those gaps is the 2 s unconditional `Wait` at the top of a recovery episode, which is a pleasing confirmation that the new tree does what section 1 says.

Mostly this is harmless, because a hard stop during a `Wait` is what you want anyway. One case was not: the first gap in run 2 opens while the wheels are still doing **0.72 m/s**, so the deceleration ramp that `max_decel` exists to

shape was replaced by the base's own brake.

I set `velocity_timeout` to **0.4 s**, which keeps every output younger than the base's 0.5 s threshold. The alternative is to raise `cmd_vel_timeout` in `diffbot_controllers.yaml`; I have not done that, both because it is outside the files I change and because it would loosen the base's own watchdog. Worth knowing that from the 2.235 m/s cap a full ramp needs 1.12 s and will still be cut short — 0.4 s covers the speeds you actually drive at.

8. Run 10, and why I did not tune for it

You have since confirmed by teleop that run 10's course was borderline impossible for a human driver with this platform's dimensions. That matters for how the data is read, so to be explicit: **I have not treated run 10 as a defect to tune against.** It is used in this report only as a stress case, for three things it shows clearly — the planner's timeout behaviour under a genuinely hard query (section 2), the command-rejection count (section 7), and eight clean recovery episodes with correct escalation (section 1).

For the record, the robot got 2.5 m in and stopped 6.37 m short. The straight line to the goal accumulated 139 → 800 lethal cells over the run, so it really was walled off, and the goal aborted with 208 at 103.2 s after the eighth recovery episode — which is `number_of_retries=8` behaving exactly as written.

The repeated contact with the bins is still worth not dismissing. The robot's centre cell never reached 253 and its footprint never touched a lethal cell in any sampled frame of any run, so the cost map did not believe it was hitting anything. On a course with 4 cm of margin, the difference between the map's view and the world is smaller than the map's own resolution.

9. What I changed

file	parameter	from	to	why
Course2027.yaml	`velocity_timeout`	1.0	**0.4**	Stops the smoother handing the base 10 commands per gap that it discards. Section 7.
Course2027.yaml	`movement_time_allowance`	20.0	**8.0**	The 20 s dead wait in run 8. Keeps a 4× margin over the slowest measured speed. Section 6.
Course2027.yaml	`terminal_checking_interval`	5000	**1000**	Makes `max_planning_time` actually binding; cuts worst-case planning latency from 869 ms. Section 2.
Course2027.yaml	`voxel_decay` (both layers)	30.0	**60.0**	Run 8's collision. Set jointly with the retry bound below. Section 3.
Course2027.yaml	local `publish_frequency`	10.0	**20.0**	Removes the publish aliasing that turned a 10 Hz loop into a 6.4 Hz topic. No extra map builds. Section 2.
recovery_nav_bt.xml	`number_of_retries` (NavigateRecovery)	8	**4**	Bounds the recovery loop so it gives up ~25 s before the map forgets. Section 3.

The `number_of_retries` comment in the tree still carried the iteration-1 rationale — "8 rather than 6 so that there are attempts left after the clears". That reason is void after the restructure, because the clear is no longer a list position. The comment now says so and gives the new reason.

Validation

Both files were checked before being handed back. `Course2027.yaml` parses and all six new values read back correctly. The tree was loaded into a real `bt_navigator` driven configure → activate → `NavigateToPose`, because Kilted builds the tree on activate and a plain configure reports success on a broken tree. It got as far as `Action server compute_path_to_pose not available`, which is the expected failure when the planner is not running — so the XML parsed and every node type resolved. A control with one `BackUp` renamed was

rejected with `Error at line 196: Node not recognized`, proving the check discriminates. The harness is `claude_tools/btcheck.sh`.

Not verified: every runtime claim in this report describes iteration 5's behaviour with iteration 5's parameters. None of the six changes above has been run on the robot.

10. Recommendations I did not act on

	recommendation	why I left it
1	**The robot should not drive into space it cannot see.** This is the real run-8 defect and the only fix that removes the cliff rather than moving it. The clean version is a short-range sensor that covers the blind cone — the Velodyne cannot see anything shorter than 0.96 m at 1 m. A software-only partial mitigation is to stop STVL's accelerated decay applying to voxels in the frustum but below the lowest ring, which is where a real obstacle goes to die.	Needs hardware, or code outside the two files I change.
2	**Lower the global `inflation_radius`.** You pinned this until lane detection works, and I have respected that. But the evidence for it changed this iteration, and the new argument does not depend on lanes at all: at 2.0 m in a 1.6 m corridor the inflation field cannot express "stay away", because there is nowhere further to go. It only flattens the cost landscape, which is what makes the plan flip (section 4) and MPPI crawl (section 5).	Your pin. The reason for the pin does not cover this new evidence, so it is worth re-deciding rather than just leaving.
3	**Goal placement needs a clearance rule.** Run 8's goal was unreachable to within its own 1.00 m tolerance because it sat about 1 m behind a bin and the robot's front is only 0.230 m ahead of `base_link`. A waypoint wants to be at least `xy_goal_tolerance` + obstacle half-depth + 0.23 m clear of anything.	Operational, not a parameter.
4	**Consider `wz_std` 0.45 → ~0.30.** The table in section 5 shows the sample cloud does not fit the corridor at any useful speed. Narrowing it should let faster samples survive the softmax.	I did not want to change MPPI's sampling in the same iteration as six other things. It would not be attributable.
5	**Revisit `max_planning_time`.** Now that the budget is actually enforced, 0.45 s is a real limit rather than a nominal one, and run 2 shows queries that wanted 0.87 s. There is room for ~0.6 s inside a 1 Hz replan cycle.	Changing the budget and the enforcement of it at once would confound the measurement. Enforcement first.
6	**Check the lattice's `grid_resolution` against the cost map.** The new file is `diff_4cm.json` and the global map is now 4 cm, which is consistent — but nav2 never validates these against each other, and a mismatch silently scales every path. Worth a one-off assertion since both numbers moved this iteration.	Flagging, not changing. It looks correct.
7	**The 200–220 ms tracking lag.** Four times the 40–55 ms measured in iteration 3, on wheel odometry as well as the gyro.	Not chased. Not obviously hurting at these speeds, but it should not have grown.

11. Tools, and one trap worth knowing about

New this iteration, all in `claude_tools/`:

tool	what it answers
`it5_bt.py`	Recovery-episode timeline from `/behavior_tree_log`: node order per episode, so the restructure can be checked at runtime.
`it5_plan.py`	Splits plan attempts by outcome and duration, so a timeout can be told apart from a genuine infeasibility.
`it5_cmdage.py`	Stamp age either side of the velocity smoother, gaps in the command stream, and how many outputs the base would reject.
`it5_corridor.py`	Max cost and lethal count on the segment to the goal, frame by frame, plus the robot's centre-cell cost. This is what caught run 8.
`it5_fpcost.py`	Max cost under the published footprint outline over a whole run.

tool	what it answers
`it5_stall.py`	How long the robot had been stationary when each recovery episode opened.
`it5_rosout.py`	Groups WARN+ log lines by pattern with counts and first/last times.
`btcheck.sh`	Loads a behaviour tree into a real `bt_navigator` and reports whether it parsed.

> **`/costmap` is not raw cost, and `/costmap_raw` is**
>
> The `OccupancyGrid` on `/costmap` is rescaled by `Costmap2DPublisher`: 254→100, 253→99, 1..252→1..98. A tool that reads it and tests `>= 253` finds nothing, silently and forever.
>
> This bit me twice while writing the new tools and produced a confidently wrong intermediate reading of run 8 — "no lethal cells on the line to the goal" — before I caught it. Both tools now read `costmap_raw`, which is the topic you added to the bag command this iteration and the reason that addition was worth asking for four times.
>
> **I checked whether this affected earlier iterations' conclusions. It did not.** Every pre-existing tool that reads `/costmap` routes it through `costmap_probe.grid_of()`, which converts back to raw cost, and `decay_audit.py` works on the 0–100 scale deliberately. The trap caught only the two tools written fresh today.

12. What to measure next

	what	why
1	Whether the iteration-1 signature returns with `voxel_decay: 60` — plans getting longer and more contorted as a run goes on.	This is the riskiest change in this iteration.
2	Whether `movement_time_allowance: 8.0` ever fires on a robot that was legitimately crawling.	The margin is 4× on paper. It wants confirming on a hard course.
3	Worst-case planning latency, now that the budget is enforced. Expect the 869 ms tail to collapse to roughly 470 ms.	Directly checkable with `it5_plan.py`.
4	Whether `number_of_retries: 4` aborts a goal that 8 would have rescued.	This is the one change with a real cost, and it will show up as a run that ends early.
5	A run on the **run-9 course**, repeated. Run 9 cleared cleanly with zero warnings and is the only clean baseline in the set.	Six parameters moved. Without a repeat of a course that already worked, there is no way to tell an improvement from a change of scenery.

On that last point, and offered as process rather than a finding: this iteration changed nine things at once on a course that also changed four times during the session. Run 8's and run 10's courses were both harder than anything before them, and run 9's was not. That makes it very hard to attribute any of it. One unchanged course, run twice, would be worth more than ten runs on ten layouts.